

app.py

This is the Streamlit application file. It displays the Verilog workflow, its code, and the workflow states.

Python file

```
import streamlit as st

st.set_page_config(
    page_title="LangGraph Verilog Workflow",
    page_icon="■"
)

st.title("■ LangGraph Verilog Workflow")

st.write(
    "Task Input → Developer → Tester → Manager Decision → Archiver"
)

# Verilog source code
verilog_code = r"""
module realtime_workflow (
    input wire      clk,
    input wire      reset,
    input wire      task_valid,
    input wire      developer_done,
    input wire      tester_done,
    input wire      manager_store,

    output reg      task_input_active,
    output reg      developer_active,
    output reg      tester_active,
    output reg      manager_active,
    output reg      archiver_active,
    output reg      workflow_done
);

    localparam TASK_INPUT      = 3'b000;
    localparam DEVELOPER      = 3'b001;
    localparam TESTER        = 3'b010;
    localparam MANAGER_DECISION = 3'b011;
    localparam ARCHIVER      = 3'b100;
    localparam DONE          = 3'b101;

    reg [2:0] state;
    reg [2:0] next_state;

    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= TASK_INPUT;
        else
            state <= next_state;
    end

    always @(*) begin
        next_state = state;

        case (state)
            TASK_INPUT: begin
                if (task_valid)
                    next_state = DEVELOPER;
                else
                    next_state = TASK_INPUT;
            end
            DEVELOPER: begin
                if (developer_done)
                    next_state = TESTER;
                else
                    next_state = DEVELOPER;
            end
            TESTER: begin
                if (tester_done)
                    next_state = MANAGER_DECISION;
                else
                    next_state = TESTER;
            end
            MANAGER_DECISION: begin
```



```
states = {
    "TASK_INPUT": "Accepts a new task.",
    "DEVELOPER": "Represents the developer stage.",
    "TESTER": "Represents the testing stage.",
    "MANAGER_DECISION": "Manager decides whether to store or start another task.",
    "ARCHIVER": "Stores the completed workflow.",
    "DONE": "Workflow completed."
}

for state, description in states.items():
    st.write(f"**{state}** {description}")

st.success("Verilog workflow loaded successfully.")
```